

Föreläsning 5

Ändrad
15 Oct
17:01

while, #define, relationsoperatorer

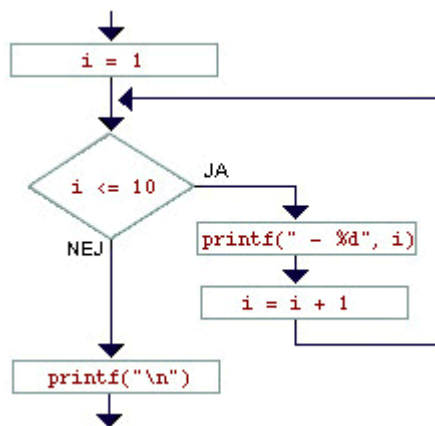
Vi kan säga åt datorn att utföra samma sats flera gånger.

Följande program skriver ut talen från ett till tio med hjälp av en `while`-sats.

```
1  #include <stdio.h>
2
3  void main()
4  {
5      int i;
6
7      i = 1;
8      while (i <= 10) {
9          printf(" - %d", i);
10         i = i + 1;
11     }
12     printf("\n");
13 }
```

De två inramade satserna (rad 9 och 10) upprepas om och om igen medan (eng. *while*) villkoret i `while`-satsen är sant. Dvs så länge som `i <= 10`.

Flödesschemat till höger anger vad som händer i `while`-loopen:



- Först testas om `i <= 10`,
- i så fall utförs de två inramade satserna.
- Därefter går datorn tillbaka och testar villkoret igen. Osv.

Om villkoret är falskt fortsätter programmet på rad 12.
Utskrift:

```
1  - 1 - 2 - 3 - 4 - 5 - 6 - 7 - 8 - 9 - 10
```

På rad 10 ändras värdet på variabeln `i`. Vi har `i` både på höger- och vänstersidan i tilldelningssatsen. Det är alltså tillåtet.

Det finns inget motsägelsefullt i det. *Först* beräknas värdet av vänstersidan. *Sedan* får vänstersidans variabel detta värde. Därvid ändras förstås värdet av högersidan, men det spelar ingen roll -- högersidan beräknas *inte* igen.

Programmet fungerar på följande sätt:

- Inledningsvis sätts `i` till 1.
- Inuti `while`-loopen ökas `i` med ett varje varv -- desutom skrivs värdet på `i` ut.
- `while`-loopen avslutas när `i` blivit 11 (eftersom 11 blir det första tal som är över tio). Därmed skrivs heller inga fler tal ut.
- Den avslutande radframmatningen (rad 12) fyller egentligen ingen funktion -- den finns bara för ordnings skull.

HI-LO

Är ni redo för ert första meningsfulla program? Det är ett spelprogram. Ok, det är inte Doom -- men det är Hi-lo!

Vi börjar med utskriften som omväxling. Så här ser en session ut. Det är användaren som gissar, och datorn som ger ledtrådar till det rätta svaret.

```

1  Nu ska vi leka hi-lo!
2  Du ska gissa mellan 1 och 100.
3  Din gissning: 50
4  För litet!
5  Din gissning: 75
6  För litet!
7  Din gissning: 85
8  För stort!
9  Din gissning: 76
9  BRA! RÄTT! DU VANN!
```

Programmet ser ut så här:

```

1  #include <stdio.h>
2  #define MIN 1
3  #define MAX 100
4
5  void main()
6  {
7      int guess = MIN - 1;
8      int correct = 76;
9
10     printf("Nu ska vi leka hi-lo!\n");
11     printf("Du ska gissa mellan %d och %d.\n", MIN, MAX);
12
13     while (guess != correct) {
14         printf("Din gissning: ");
15         scanf("%d", &guess);
16         if (guess < correct)
17             printf("För litet!\n");
18         else if (guess > correct)
19             printf("För stort!\n");
20     }
21     printf("BRA! RÄTT! DU VANN!\n");
22 }
```

Först några nya begrepp i programmet. Sen återvänder vi till programmets idé.

- På raderna 2--3 definieras *konstanter*. *Konstanter* kan inte byta värde, till skillnad från variabler. Rad 2 och 3 kallar vi *konstantdefinitioner*. Texten `#define` måste stå först i raden, annars ser inte datorn konstantdefinitionen.

Vi lägger konstantdefinitionerna högst upp i programmet. Vill du lägga dem någon annanstans måste du ha ett väldigt gott skäl.

Det korrekta namnet är inte *konstant* utan *makro* (eng. *macro*). Och rad 2 och 3 är *makrodefinitioner*.

Det finns fler möjligheter hos makron än vad ni får lära er på denna kurs. Makron används dels som konstanter, dels som förkortningar. Den som är intresserad kan studera kapitel 9.

■ **Nästa nyhet:** Variablerna `guess` och `correct` får sina första värden redan i deklarationen på raderna 7 och 8.

Denna möjlighet finns av bekvämlighetsskäl. Man skulle ju lika gärna kunna *initiera* dem (som det heter) i de första raderna i programmet.

■ **Nästa nyhet:** Villkoret ser märkligt ut: `!=`. Det är detsamma som `#`. Mer om villkor strax!

■ **Nästa nyhet:** `else`-delen innehåller en ny `if`-sats. Egentligen borde kanske satsen layoutas på följande sätt:

```
16         if (guess < correct)
17             printf("För litet!\n");
18         else
19             if (guess > correct)
20                 printf("För stort!\n");
```

Men man brukar layouta som jag gjort när `else`-delen bara består av en ny `if`-sats. Det är nämligen vanligt med en lång kedja av `else-if`-satser. (Se exempel i [kursboken](#), sid 103) Om man fortsatte skjuta in varje rad ytterligare några steg, skulle man snart hamna långt ut i marginalen.

■ **Nästa nyhet:** Det finns inga fler nyheter! Du har fått lära dig allt som behövs för att kunna förstå hur detta program fungerar!

Nu ska du *skrivbordstesta* programmet! Utför programmet steg för steg -- precis som datorn måste göra! Om du behöver: anteckna på papper de värden som de olika variablerna har för tillfället.

Stega rad för rad. Snurra några varv i `while`-loopen. Jämför med utskriften.

Relationsoperatorer

Följande *relationsoperatorer* finns:

- `==` som är =
- `>` som är >
- `<` som är <
- `<=` som är ≤
- `>=` som är ≥
- `!=` som är ≠

Lägg märke till att likhet skrives med *två* `=`-tecken! Om du vill avgöra om `i` och `j` är lika, ska du alltså skriva `if`

! `(i==j) . . . .` Inget annat. **!**

Du får ingen hjälp av kompilatorn om du gör fel. Det beror på att `if(i=j) ..` är ett fullt tillåtet uttryck! Vi återkommer snart till vad uttrycket betyder.

Nästa lektion

